Anexo. Archivos del prototipo y evidencias

SSGUI · Sistema de Solicitud de Gestión de Usuarios Interno · ANDINA S.A.S. (empresa ficticia; todos los datos son ficticios)
José Bedoya, Carlos Ruiz, Luis Cerón y Cristian Caro · Institución Universitaria Politécnico Grancolombiano · Taller de Desarrollo Seguro de Software · Grupo G01
Docente: José Eduardo Patiño Santafé · 9 de noviembre de 2026

1. Propósito

Este anexo reúne y ubica los archivos que componen el prototipo y las evidencias de su desarrollo, pruebas, implementación y solución. Todo está en el repositorio del proyecto y es reproducible con los comandos indicados. Las rutas son relativas a la raíz del repositorio.

2. Código fuente del prototipo

El prototipo es una aplicación web con arquitectura hexagonal. El núcleo (dominio y aplicación) no depende de ningún adaptador, lo que permite probarlo sin red ni base de datos.

Tabla 1. Código fuente por capa

Capa	Ruta	Contenido	Tamaño
Dominio	`src/jml/dominio/`	Entidades y reglas puras: solicitud, ticket, cuenta, perfiles, auditoría encadenada	17 archivos, ~2 984 líneas
Aplicación	`src/jml/aplicacion/`	Casos de uso y puertos (contratos)	17 archivos, ~2 907 líneas
Adaptadores	`src/jml/adaptadores/`	Web (Django), trabajador, cliente de GLPI, persistencia, identidad, servicio de cuentas	46 archivos, ~5 417 líneas
Configuración	`src/config/`	Ajustes de Django; lee los secretos de `/run/secrets`	5 archivos, ~276 líneas
Infraestructura	`infra/`	`compose.yaml`, BunkerWeb, reportería, GLPI como código, scripts	—

3. Pruebas (demostración del funcionamiento)

Tabla 2. Suites de prueba y cómo ejecutarlas

Qué demuestra	Ruta / comando
Reglas de negocio, casos de uso, contratos, web, seguridad	`tests/` (99 archivos de prueba); `make verificar`
Núcleo, aplicación, contratos, escenarios, adaptadores	`uv run pytest -q tests/dominio tests/aplicacion tests/contratos tests/escenarios tests/adaptadores`
Integración con PostgreSQL 17	`bash infra/scripts/integracion-local.sh`
Flujos de extremo a extremo con navegador, por el WAF, hasta GLPI	`JML_GLPI_DE_PRUEBAS=si bash infra/scripts/e2e-interfaz.sh`
Pruebas de ataque (MitM, inyección SQL)	`bash auditoria/scripts/mitm-interceptacion.sh`, `sqli-manual.sh`, `sqlmap-nivel5.sh`
Auditoría funcional por rol, a través del WAF	`auditoria/scripts/auditoria-funcional.py`, `auditoria-crud-usuarios.py`
Rendimiento (300 colaboradores, 20 usuarios)	`bash infra/scripts/carga.sh`

4. Implementación (cómo se levanta y opera)

Tabla 3. Instalación y operación

Procedimiento	Ruta / comando
Levantar todo el sistema	`docker compose -f infra/compose.yaml -f infra/compose.local.yaml --profile glpi up -d --build`
Crear tablas y cargar datos ficticios	`... run --rm migrar` y `... exec web python manage.py semillas`
Configurar GLPI como código	`... run --rm configurar-glpi`
Comprobar el WAF	`infra/waf/pruebas.sh app.localhost` y `... glpi.localhost`

5. Manuales, procedimientos e instructivos

Tabla 4. Documentación del prototipo

Documento	Ruta	Contenido
Manual de usuario	`docs/manual-usuario.md`	Ingreso, solicitudes, aprobación, tickets, administración de usuarios
Guía de instalación local	`docs/probar-localmente.md`	Diez pasos para levantar el sistema en un equipo
Operación	`docs/operacion.md`	Contenedores, secretos, respaldo y restauración
Especificación de requisitos (SRS)	`docs/SRS.md`	Requisitos, invariantes, amenazas, criterios de aceptación
Registro de decisiones (ADR)	`docs/decisiones.md`	Decisiones de arquitectura, incluidas ADR-035 a ADR-037
Plan de auditoría	`docs/PLAN_AUDITORIA.md`	Las diez fases de la auditoría

6. Entregables de Desarrollo Seguro

Tabla 5. Documentos de la asignatura

N.º	Documento	Ruta
1	Resultados de pruebas funcionales	`docs/Desarrollo Seguro/1-Resultados-de-pruebas-funcionales.pdf`
2	Resultados de pruebas de vulnerabilidades	`docs/Desarrollo Seguro/2-Resultados-de-pruebas-de-vulnerabilidades.pdf`
3	Análisis de captura de tráfico	`docs/Desarrollo Seguro/3-Analisis-de-captura-de-trafico.pdf`
4	Propuesta de controles de seguridad	`docs/Desarrollo Seguro/4-Propuesta-de-controles-de-seguridad.pdf`
5	Pruebas de ataque MitM y SQL Injection	`docs/Desarrollo Seguro/5-Pruebas-de-ataque-MitM-y-SQLi.pdf`

7. Evidencias crudas de las pruebas (solución verificada)

Cada medición conserva su comando exacto, su salida con los secretos enmascarados, la versión de la herramienta, el *commit* y la fecha, y su *hash* en un manifiesto que un verificador comprueba antes de la entrega.

Tabla 6. Carpetas de evidencia

Categoría	Ruta	Ejemplos
Funcionales	`auditoria/evidencia/funcionales/`	Suite, cobertura, integración, E2E, mutación
Estáticos	`auditoria/evidencia/estaticos/`	ruff, bandit, mypy, gitleaks, Trivy
Vulnerabilidades	`auditoria/evidencia/vulnerabilidades/`	sqlmap, ZAP, nmap, testssl, **DS5-SQLI-**, ***DS5-MITM-***
Tráfico	`auditoria/evidencia/trafico/`	Capturas por caso de uso
Disponibilidad	`auditoria/evidencia/disponibilidad/`	Respaldo, detección de caídas, carga
Autorización	`auditoria/evidencia/autorizacion/`	Matriz de autorización
Controles positivos	`auditoria/evidencia/controles-positivos/`	Defectos sembrados detectados

Verificación de integridad de la evidencia:

```
sha256sum -c auditoria/manifiesto.sha256
python docs/entregables/auditoria/verificar-entrega.py docs/entregables/auditoria
```

8. Video(s) de demostración

El video de demostración está en `docs/Desarrollo Seguro/video/demostracion-ssgui.mp4` (57 s). Recorre, con fotogramas reales de la aplicación capturados a través del WAF y tarjetas de resultado: la implementación, el inicio de sesión, la creación de eventos, la consulta de tickets, el dashboard, la administración de usuarios y las dos pruebas de ataque (inyección SQL y hombre en el medio) con su veredicto. Se regenera con:

```
bash auditoria/scripts/mitm-interceptacion.sh   # (opcional) refrescar resultados de ataque
uv run --with playwright --with pillow python auditoria/scripts/construir-video-demo.py <cookie> <salida>
```

El guion reproducible —qué mostrar y con qué comando— es además esta misma lista de tablas: cada fila es un paso verificable en vivo.